ERD Essentials — Entiteiten, Attributen, Sleutels & Relaties (NL)

Een compacte, beginnersvriendelijke referentie gebaseerd op de geüploade slides, met extra uitleg, voorbeelden en kant■en■klare SQL■snippets.

1) Van data naar database
- **Data** zijn ruwe feiten; **informatie** is data met betekenis.
- Een **database** (meestal relationeel) bewaart data in **tabellen** (rijen = records, kolommen = attributen).
- Basisregels van het relationele model:
  • Elke tabel heeft een **primaire sleutel (PK)** (uniek, niet■NULL).
  • Een cel bevat precies één waarde van het columntype.
  • Kolommen hebben een consistente betekenis.

Waarom dit telt: Duidelijke sleutels + consistente structuur maken queries betrouwbaar en de prestaties voorspelbaar.

2) Modellering in lagen
- **Conceptueel model**: enkel kernentiteiten en relaties (M:N mag nog).
- **Logisch model**: voegt attributen en kandidaatsleutels toe; M:N kun je al vervangen door associatietabellen.
- **Fysiek model**: concreet voor een RDBMS. Inclusief datatypes, PK/FK/AK, indexen, NULL vs NOT NULL en referential actions (bv. `ON DELETE`).

Tip: Werk top■down (conceptueel → logisch → fysiek). Spring niet te vroeg in SQL.

3) Naamconventies (aanbevolen)
- **Tabellen**: enkelvoud, hoofdletter vooraan (bv. `Klant`, `Product`, `Order`).
- **Primaire sleutel**: `id` (integer, surrogaat, betekenisloos).
- **Attributen**: `camelCase` (bv. `voornaam`, `orderDatum`).
- **Foreign keys**: `<enkelvoudVerwezenTabel>Id` (bv. `klantId`, `productId`).

Vuistregel: Kies één stijl en pas die overal consequent toe.

4) Entiteiten & attributen
- **Entiteit**: iets waarover je info bewaart (Klant, Product, Order…).
- **Attribuut**: eigenschap van een entiteit.
- **Splits samengestelde gegevens**: bv. `voornaam` + `achternaam` (niet één `naam`).
- **Vermijd procesdata**: sla **geen** leeftijd op; bewaar `geboorteDatum` en bereken leeftijd.

5) Sleutels
5.1 Primaire sleutel (PK)
- Identificeert een rij uniek; nooit NULL; stabiel.
- Liefst een **surrogaatsleutel** `id` (betekenisloos, nooit bewerken).

5.2 Kandidaat- en alternatieve sleutels
- **Kandidaatsleutels**: alle kolommen/kolomsets die PK zouden kunnen zijn (bv. `studentenNummer`, soms `email`).
- **Alternatieve sleutels (AK)**: niet■gekozen kandidaten; afdwingen met `UNIQUE`.

5.3 Waarom surrogaatsleutels?
- Natuurlijke sleutels kunnen wijzigen (fouten, businesswijzigingen).
- Integers zijn compact en snel voor joins; minder cascade■updates.

6) Relaties & Foreign Keys (FK)
- **Foreign Key** verbindt een kindrij met de PK van de ouder.
- **Cardinaliteit**:
  • **1:1**: zeldzaam—vaak samenvoegen tot één tabel tenzij er een goede reden is.
  • **1:N**: meest voorkomend. De FK staat aan de **N (*)■kant**.
  • **M:N**: niet rechtstreeks; gebruik een **associatietabel** met twee FK’s.
- **Optionaliteit (aard)**:
  • **1** (verplicht) → FK is `NOT NULL`.
  • **0** (optioneel) → FK **mag `NULL`** zijn.
- **Referential actions**: `ON DELETE RESTRICT | CASCADE | SET NULL`.

Gouden regel: *Zet de FK op de *■kant van een 1:N■relatie.*

7) Voorbeeld — Shop■schema
7.1 Entiteiten
- `Klant(id, voornaam, achternaam, email)`
- `Product(id, naam, prijs)`
- `Order(id, orderDatum, klantId)`
- `OrderProduct(orderId, productId, aantal)` ← associatietabel (M:N)

7.2 SQL (generiek/portable)

```
-- Tabel: Klant
CREATE TABLE Klant (
  id        INT PRIMARY KEY,
  voornaam   VARCHAR(80)  NOT NULL,
  achternaam VARCHAR(120) NOT NULL,
  email      VARCHAR(255) NOT NULL UNIQUE  -- AK
);

-- Tabel: Product
CREATE TABLE Product (
  id    INT PRIMARY KEY,
  naam  VARCHAR(120) NOT NULL,
  prijs DECIMAL(10,2) NOT NULL
);

-- Tabel: Order (FK staat aan de *-kant)
CREATE TABLE "Order" (
  id        INT PRIMARY KEY,
  orderDatum DATE NOT NULL,
  klantId    INT  NOT NULL,
   CONSTRAINT fk_order_klant
     FOREIGN KEY (klantId) REFERENCES Klant(id)
    ON DELETE RESTRICT  -- kies bewust: RESTRICT | CASCADE | SET NULL
);

-- Associatie: Order <-> Product
CREATE TABLE OrderProduct (
  orderId   INT NOT NULL,
  productId INT NOT NULL,
  aantal    INT NOT NULL CHECK (aantal > 0),
   PRIMARY KEY (orderId, productId),
   CONSTRAINT fk_op_order
     FOREIGN KEY (orderId) REFERENCES "Order"(id) ON DELETE CASCADE,
   CONSTRAINT fk_op_product
     FOREIGN KEY (productId) REFERENCES Product(id) ON DELETE RESTRICT
);
```

7.3 Handige queries

```
-- 1) Alle orders met klantnaam
SELECT o.id, o.orderDatum, k.voornaam, k.achternaam
FROM "Order" o
JOIN Klant k ON k.id = o.klantId
ORDER BY o.orderDatum DESC;

-- 2) Orderregels met product en subtotaal
SELECT op.orderId, p.naam, op.aantal, (op.aantal * p.prijs) AS subtotaal
FROM OrderProduct op
JOIN Product p ON p.id = op.productId
ORDER BY op.orderId;

-- 3) Totaal per order
```

```
SELECT op.orderId,
     SUM(op.aantal * p.prijs) AS totaal
FROM OrderProduct op
JOIN Product p ON p.id = op.productId
GROUP BY op.orderId
ORDER BY op.orderId;
```

Check: Waarom is `klantId` `NOT NULL` in `Order`? (Hint: elke order hoort bij precies één klant.)

8) Optionele relatie — Book ↔ Series (0..1)

```
-- Tabel: Series (optionele ouder voor Book)
CREATE TABLE Series (
  id   INT PRIMARY KEY,
  name VARCHAR(200) NOT NULL
);

-- Tabel: Book (optionele FK naar Series)
CREATE TABLE Book (
  id       INT PRIMARY KEY,
  title    VARCHAR(200) NOT NULL,
  seriesId INT NULL,   -- optioneel (aard = 0)
    CONSTRAINT fk_book_series
      FOREIGN KEY (seriesId) REFERENCES Series(id)
     ON DELETE SET NULL
);
```
---

9) Klassieke M:N — Student ↔ Vak

```
-- Tabel: Student
CREATE TABLE Student (
  id            INT PRIMARY KEY,
  voornaam        VARCHAR(80)  NOT NULL,
  achternaam      VARCHAR(120) NOT NULL,
  studentenNummer  VARCHAR(20)  NOT NULL UNIQUE -- AK
);

-- Tabel: Vak
CREATE TABLE Vak (
  id   INT PRIMARY KEY,
  naam VARCHAR(120) NOT NULL
);

-- Associatietabel: StudentVak
CREATE TABLE StudentVak (
  studentId INT NOT NULL,
  vakId     INT NOT NULL,
  jaar      SMALLINT NOT NULL,
    PRIMARY KEY (studentId, vakId),
    FOREIGN KEY (studentId) REFERENCES Student(id) ON DELETE CASCADE,
    FOREIGN KEY (vakId)     REFERENCES Vak(id)     ON DELETE RESTRICT
);
```
---

10) Checklists
ERD■checklist
[ ] Elke tabel heeft **PK** (`id INT`, betekenisloos).
[ ] Bepaal **AK's** (natuurlijke uniciteit) en zet `UNIQUE`.
[ ] **FK staat aan de *■kant** (1:N) en heeft een duidelijke naam.
[ ] **M:N** altijd via **associatietabel** met **samengestelde PK**.
[ ] Optionaliteit vertaalt naar `NULL` vs `NOT NULL` op de FK.
[ ] Vermijd procesdata; splits samengestelde velden.
[ ] Consequente naamgeving en passende datatypes.

Keuzehulp referential actions
- **RESTRICT**: voorkomen van weeskinderen; verwijderen blokkeren bij afhankelijke rijen.
- **CASCADE**: afhankelijke rijen automatisch mee verwijderen.
- **SET NULL**: relatie is optioneel; bij verwijderen wordt de FK `NULL`.

11) Veelgemaakte fouten
- **Betekenisvolle PK's** (RRN, serienummer) → kwetsbaar voor wijziging → gebruik `id`.
- **Duplicatie** (berekenbare/afleidbare waarden) → normaliseren.
- **FK aan de verkeerde kant** → volg de *■kant■regel.
- **Onnodige directe relaties** als er al een impliciet pad bestaat.

12) Mini■oefening
1) Modelleer **Klas (1) — (*) Student**. Waar komt de FK en is die `NULL` of `NOT NULL` als elke student verplicht in precies één klas zit?

Appendix — Snelle notatie (ASCII)
Klant (1) ■■■< (N) Order
  id PK id PK
  klantId FK → Klant.id

Student (N) >■■■< (N) Vak via StudentVak
StudentVak PK(studentId, vakId)